AgroManage – Complete Project & Viva Guide

Full Screens Catalog, Project Architecture, and Live Coding Handbook for Mobile Application Development (MAD) Mid Lab Assessment

Course: Mobile Application Development (MAD)	**Tech Stack:** Flutter 3.47 / Dart 3.13
Project: AgroManage (Kisan Agro Center)	**State Management:** ChangeNotifier (AppState)
Platform: Android (API 34/36, minSdk 21)	**Repository:** github.com/ahmadd26/AgroManage

1. Project Architecture & File Organization

In Flutter, code is organized inside the lib/ folder. Here is the directory layout of AgroManage and the role of each directory:

Directory / File	Purpose & Contents
`lib/main.dart`	**Application Entry Point.** Contains main(), runApp(), initializes AgroManageApp, and manages the initial routing (Splash -> Login -> MainShell).
`lib/models/`	**Data Entities / Blueprints.** Plain Dart classes: Product, Customer, SaleInvoice, InstallmentPlan, Supplier, and Expense.
`lib/state/app_state.dart`	**Global State Management.** Extends ChangeNotifier. Holds in-memory lists, mock data, and business logic (adding products, invoices, recording payments, deducting stock).
`lib/theme/app_theme.dart`	**App Styling & Colors.** Agriculture green palette (#2E7D32), typography, button themes, card themes, and currency formatters (PKR).
`lib/widgets/custom_widgets.dart`	**Reusable UI Components.** MetricCard (dashboard stats), StatusBadge (colored tags), and SectionHeader.
`lib/screens/`	**UI Pages (22 screens).** All interactive screens: Dashboard, POS, Products, Customers, Khata Installments, Suppliers, Expenses, Reports, etc.

2. Essential Flutter Concepts for Live Coding

The examiner may ask you to recreate or modify code live during your viva. Below are the core concepts you must know:

A. How to Change the Starting Screen

The starting screen is controlled by the home: property of MaterialApp in lib/main.dart:

```
MaterialApp(
  title: 'AgroManage',
  theme: AppTheme.lightTheme,
  home: SplashScreen(onFinish: () { ... }), // <-- CHANGE THIS to test any screen!
  // e.g. home: ProductsScreen(state: _appState, onSelectProduct: ...),
);
```

B. Creating a StatelessWidget vs StatefulWidget

• **StatelessWidget:** Used when the screen only displays data and does NOT change dynamically (e.g. static card, info dialog).
• **StatefulWidget:** Used when user interaction changes data dynamically (e.g. form fields, checkboxes, counters, timers).

```
// 1. STATELESS WIDGET:
class MyInfoCard extends StatelessWidget {
  final String title;
  const MyInfoCard({super.key, required this.title});

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Text(title, style: const TextStyle(fontSize: 16)),
      ),
    );
  }
}

// 2. STATEFUL WIDGET:
class CounterScreen extends StatefulWidget {
  const CounterScreen({super.key});
  @override
  State<CounterScreen> createState() => _CounterScreenState();
}

class _CounterScreenState extends State<CounterScreen> {
  int _count = 0;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Counter')),
      body: Center(child: Text('Count: $_count')),
      floatingActionButton: FloatingActionButton(
        onPressed: () {
          setState(() { _count++; }); // setState tells Flutter to re-render build()!
        },
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

C. How to Add an Image from Scratch

Step 1: Create folder assets/images/ in project root and put your image there (e.g. logo.png).

Step 2: Register the asset in pubspec.yaml under flutter::

```
flutter:
  assets:
    - assets/images/
```

Step 3: Run flutter pub get.

Step 4: In your Dart file, display it with Image.asset() or Image.network():

```
// Display Local Asset Image:
Image.asset('assets/images/logo.png', width: 100, height: 100, fit: BoxFit.cover)

// Display Network Image (from URL):
Image.network('https://example.com/fertilizer.jpg', width: 100, height: 100)
```

D. How to Navigate Between Screens

Flutter manages navigation using a stack. Navigator.push adds a screen; Navigator.pop closes it:

```
// 1. Open a new screen (Push):
Navigator.push(context, MaterialPageRoute(builder: (context) => AddProductScreen(state: appState)));

// 2. Go back to previous screen (Pop):
Navigator.pop(context);

// 3. Replace screen without back button (Login -> MainShell):
Navigator.pushReplacement(context, MaterialPageRoute(builder: (context) => MainShell(state:
appState)));
```

E. How to Import Files & Screens

```
// Relative Import (based on current folder depth):
import '../models/product.dart';          // Up one folder to models
import '../screens/dashboard_screen.dart'; // Up one folder to screens
import 'package:flutter/material.dart';     // Flutter core framework

// Package Import (using app name from pubspec.yaml):
import 'package:agro_manage/models/customer.dart';
```

F. Form Validation & Text Controllers

```
final _formKey = GlobalKey<FormState>();
final _nameCtrl = TextEditingController();

@override
void dispose() {
  _nameCtrl.dispose(); // Always dispose controllers to prevent memory leaks!
  super.dispose();
}

// In build():
Form(
  key: _formKey,
  child: Column(children: [
    TextFormField(
      controller: _nameCtrl,
      decoration: const InputDecoration(labelText: 'Customer Name *'),
      validator: (v) => (v == null || v.trim().isEmpty) ? 'Name required' : null,
    ),
    ElevatedButton(
      onPressed: () {
        if (_formKey.currentState!.validate()) {
          print('Entered Name: ${_nameCtrl.text}');
        }
      },
      child: const Text('Save'),
    ),
  ]),
)
```

3. Complete Catalog of All 22 AgroManage Screens

Detailed reference for each screen in the project, what it does, and what questions you may be asked:

01. Splash Screen

`lib/screens/splash_screen.dart`

Purpose & Logic: Displays the brand logo (leaf emblem in green gradient container), 'AgroManage' typography, and center location pill ('Kisan Agro Center · Sahiwal'). Plays a smooth 1000ms entrance animation and holds for 4 seconds before automatically routing to the Login Screen.

Key Widgets Used: StatefulWidget, AnimationController, CurvedAnimation, Timer, Gradient Container.

Viva Live Coding Tip: Q: 'How does it transition automatically?' -> A: 'Inside initState(), a Timer(Duration(seconds: 4)) calls the widget.onFinish callback.'

02. Login Screen

`lib/screens/login_screen.dart`

Purpose & Logic: Authentication portal with role selector (Shop Owner / Admin, Sales Manager, Accountant). Validates username and password against mock credentials (admin@agromanage.pk / 123456) and logs the user in.

Key Widgets Used: TextEditingController, Form, DropdownButtonFormField, password obscuring toggle.

Viva Live Coding Tip: Q: 'How do you toggle password visibility?' -> A: 'A boolean _obscurePassword in state is toggled using setState(() => _obscurePassword = !_obscurePassword).'

03. Main Shell (Navigation Hub)

`lib/screens/main_shell.dart`

Purpose & Logic: The master container that wraps the entire app after login. Renders the top AppBar (shop branding, dark mode toggle, notification bell with badge), the active screen body via AnimatedSwitcher, and the persistent BottomNavigationBar.

Key Widgets Used: Scaffold, BottomNavigationBar, NavigationRail (responsive wide screens), AnimatedSwitcher.

Viva Live Coding Tip: Q: 'How does navigation work without destroying state?' -> A: 'MainShell holds _currentRoute string. Changing the tab updates _currentRoute via setState() and renders the corresponding screen.'

04. Executive Dashboard

`lib/screens/dashboard_screen.dart`

Purpose & Logic: The executive home screen. Calculates and displays 4 real-time business KPIs: Today's Sales, Cash Collected, Pending Farmer Khata Dues, and Daily Expenses. Surfaces low stock and expiring chemical alerts with 1-tap reordering.

Key Widgets Used: MetricCard, Alert Cards, recent invoices ListView, FloatingActionButton for quick POS.

Viva Live Coding Tip: Q: 'Where do these KPI numbers come from?' -> A: 'AppState computes them dynamically using getters like state.todayGrossSales and state.totalPendingBalance.'

05. Point of Sale (POS) Billing

`lib/screens/pos_screen.dart`

Purpose & Logic: The primary sales register. Allows selecting a farmer customer (instantly shows their existing credit balance), adding products from inventory with quantity multipliers, selecting payment mode (Cash, Bank, Khata), and creating an invoice.

Key Widgets Used: Dynamic cart List, DropdownButtonFormField, Form, stock validation, SnackBar feedback.

Viva Live Coding Tip: Q: 'What happens when an invoice is completed?' -> A: 'AppState.recordSaleInvoice() is called. It deducts the sold quantity from each product stock and adds the unpaid amount to customer Khata.'

06. Products Catalog

`lib/screens/products_screen.dart`

Purpose & Logic: Interactive inventory browser. Lists all seeds, fertilizers, and pesticides with stock level badges. Supports search by product name/crop target and category chips filtering (Pesticides, Fertilizers, Seeds, Machinery).

Key Widgets Used: ListView.builder, TextField search filter, ChoiceChip, StatusBadge, FloatingActionButton.

Viva Live Coding Tip: Q: 'How does real-time search work?' -> A: 'The search TextField onChanged listener updates _searchQuery and filters the list with .where((p) => p.name.toLowerCase().contains(query)).'

07. Add Product Screen

`lib/screens/add_product_screen.dart`

Purpose & Logic: Full-featured product entry form. Fields: Product name, Brand/Manufacturer, Category, Packaging Unit (Bag, Liter, Kg), Purchase Cost, Retail Price, Initial Stock, Minimum Threshold, Batch Number, and Expiry Date picker.

Key Widgets Used: Form, GlobalKey, TextFormField with numeric keyboards, showDatePicker dialog.

Viva Live Coding Tip: Q: 'How do you pick an expiry date?' -> A: 'Using Flutter built-in showDatePicker(context: context, initialDate: ..., firstDate: ..., lastDate: ...).'

08. Product Detail Screen

`lib/screens/product_detail_screen.dart`

Purpose & Logic: Detailed profile for a specific chemical or seed item. Displays pricing margins, gross profit per unit, batch number, active ingredient formula, stock status indicator, and quick action to record a supplier purchase.

Key Widgets Used: Card, StatusBadge, custom metric chips, price calculation helpers.

Viva Live Coding Tip: Q: 'How is profit margin calculated?' -> A: 'margin = (retailPrice - purchasePrice) / retailPrice * 100.'

09. Customers Directory

`lib/screens/customers_screen.dart`

Purpose & Logic: Directory of all farmers registered with the shop. Shows farmer name, village/address, phone number, acreage, and their outstanding Khata credit balance in colored badges.

Key Widgets Used: ListView.builder, search filter, Customer Card, phone dialer intent launcher.

Viva Live Coding Tip: Q: 'How are customers with high debt highlighted?' -> A: 'If getCustomerBalance(id) > 50,000, StatusBadge type is red ("Overdue"); if 0, green ("Clear").'

10. Add Customer Screen

`lib/screens/add_customer_screen.dart`

Purpose & Logic: Farmer registration form. Captures farmer's full name, mobile number, village/tehsil address, CNIC number, land size in acres, and primary crop cultivated (Wheat, Cotton, Rice, Sugarcane).

Key Widgets Used: TextFormField with validation, land size numeric input, AppState.addCustomer.

Viva Live Coding Tip: Q: 'How is CNIC validated?' -> A: 'Validator checks regex or length of 13 digits (without dashes) or 15 characters.'

11. Customer Detail & Ledger

`lib/screens/customer_detail_screen.dart`

Purpose & Logic: Comprehensive farmer Khata account ledger. Displays total lifetime purchases, cash paid to date, remaining balance, and historical list of all invoices issued to this farmer with click-through to invoice details.

Key Widgets Used: Nested ListView, Card, balance indicator banner, 'Record Payment' direct button.

Viva Live Coding Tip: Q: 'Where is the customer invoice history retrieved?' -> A: 'AppState filters invoices using state.invoices.where((inv) => inv.customer.id == customerId).'

12. Sales Invoices Screen

`lib/screens/invoices_screen.dart`

Purpose & Logic: Master invoice history. Lists all issued bills with invoice number (e.g. INV-2026-001), date, farmer name, item count, total amount, and payment status (Paid, Partial, Khata Due).

Key Widgets Used: ListView.builder, StatusBadge, date formatting via AppTheme.formatDate.

Viva Live Coding Tip: Q: 'How do you format timestamps in Flutter?' -> A: 'AppTheme.formatDate(DateTime dt) formats day, month name, and year.'

13. Invoice Detail Screen

`lib/screens/invoice_detail_screen.dart`

Purpose & Logic: Full receipt breakdown for a completed sale. Displays line items table (Product, Quantity, Unit Price, Total), payment breakdown (Cash paid vs Khata debt), and a 'Print / Share Receipt' action.

Key Widgets Used: Table / Column layout, Divider lines, receipt styling, customer summary card.

Viva Live Coding Tip: Q: 'Can you show me how line totals are added?' -> A: 'inv.items.fold(0.0, (sum, item) => sum + (item.quantity * item.unitPrice)).'

14. Record Payment Screen

`lib/screens/record_payment_screen.dart`

Purpose & Logic: Farmer debt collection screen. Selects an invoice or farmer with pending balance, inputs the payment amount received, chooses payment channel (Cash, JazzCash, EasyPaisa, Bank), and updates the ledger.

Key Widgets Used: DropdownButtonFormField, numeric payment validator, AppState.recordPayment.

Viva Live Coding Tip: Q: 'What happens if a farmer pays more than their dues?' -> A: 'The validator checks if amount > remainingBalance and shows an error preventing overpayment.'

15. Installments (Khata Plans)

`lib/screens/installments_screen.dart`

Purpose & Logic: Manages seasonal farmer credit agreements. Lists structured installment schedules tied to crop harvesting seasons, overdue payment alerts, and penalty tracking.

Key Widgets Used: ExpansionTile / Card, overdue date comparison, AppTheme.formatCurrency.

Viva Live Coding Tip: Q: 'How are overdue installments detected?' -> A: 'plan.dueDate.isBefore(DateTime.now()) && plan.paidAmount < plan.dueAmount.'

16. Agrochemical Suppliers

`lib/screens/suppliers_screen.dart`

Purpose & Logic: Directory of certified seed, fertilizer, and pesticide manufacturers (e.g. Engro Fertilizers, Bayer CropScience, FMC, Syngenta). Shows sales representative contacts and total stock supplied.

Key Widgets Used: ListTile, CircleAvatar branding, phone call action.

Viva Live Coding Tip: Q: 'What model represents a vendor?' -> A: 'The Supplier model with fields: id, name, representative, phone, city, productsSupplied.'

17. Purchases (Stock In)

`lib/screens/purchases_screen.dart`

Purpose & Logic: Log of wholesale inventory restocks. Tracks purchase order numbers (PO-101), supplier name, shipment arrival date, and total wholesale invoice cost.

Key Widgets Used: ListView.builder, StatusBadge, wholesale invoice summary.

Viva Live Coding Tip: Q: 'How does purchasing affect inventory?' -> A: 'Recording a purchase increases the stock count of the corresponding Product in AppState.'

18. New Purchase Order

`lib/screens/new_purchase_screen.dart`

Purpose & Logic: Restock entry form. Selects a supplier, chooses the product received, inputs wholesale quantity and bulk purchase price, and updates stock counts automatically.

Key Widgets Used: DropdownButtonFormField, Form, numeric multipliers, AppState.recordPurchaseOrder.

Viva Live Coding Tip: Q: 'How is new stock added to existing stock?' -> A: 'AppState finds the product by id and runs product.stock += newQuantity; notifyListeners().'

19. Shop Expenses Screen

`lib/screens/expenses_screen.dart`

Purpose & Logic: Daily operational cost manager. Logs shop expenses categorized by Utility Bills (Tube-well, WAPDA), Generator Diesel, Store Rent, Staff Daily Wages, and Freight/Transport.

Key Widgets Used: showDialog popup entry, category dropdown, expense list, total expenses KPI.

Viva Live Coding Tip: Q: 'How do you create a popup dialog in Flutter?' -> A: 'Using showDialog(context: context, builder: (context) => AlertDialog(...)).'

20. Financial Reports Screen

`lib/screens/reports_screen.dart`

Purpose & Logic: Business intelligence & summaries. Shows Net Profit = (Gross Revenue - Cost of Goods - Shop Expenses), cash vs credit breakdown percentages, and top 5 best-selling chemicals.

Key Widgets Used: Summary cards, percentage progress bars (LinearProgressIndicator), mathematical calculations.

Viva Live Coding Tip: Q: 'How is Net Profit calculated in the code?' -> A: 'final netProfit = totalSales - totalCostOfGoods - totalExpenses.'

21. Alerts & Notifications

`lib/screens/notifications_screen.dart`

Purpose & Logic: Safety and inventory alert center. Categorizes urgent alerts into: 1) Low Stock warnings, 2) Agrochemicals expiring within 90 days, and 3) Overdue farmer installment accounts.

Key Widgets Used: Card, StatusBadge, quick action buttons ('Reorder Stock', 'Call Farmer').

Viva Live Coding Tip: Q: 'How do you filter expiring chemicals?' -> A: 'state.products.where((p) => p.expiryDate.difference(DateTime.now()).inDays <= 90).'

22. Settings & Preferences

`lib/screens/settings_screen.dart`

Purpose & Logic: App settings screen. Allows toggling Dark Mode / Light Mode, changing shop business details (Shop Name, License No, Phone), exporting data, and user logout.

Key Widgets Used: SwitchListTile, AppState.toggleThemeMode(), logout callback.

Viva Live Coding Tip: Q: 'How does Dark Mode switch instantly across all screens?' -> A: 'AppState calls notifyListeners(), causing AnimatedBuilder in main.dart to rebuild with AppTheme.darkTheme.'

4. Top 15 Viva Questions & Direct Answers

Read these answers carefully so you can speak confidently during evaluation:

1. What is the difference between Hot Reload and Hot Restart in Flutter?

Answer: Hot Reload (r) injects updated source code files into the running Dart VM without destroying the application state (takes ~1 sec). Hot Restart (R) destroys the current state and restarts the app from main() (takes ~3 sec). Full rebuild is required when adding native assets or plugins.

2. What is the purpose of BuildContext in Flutter?

Answer: BuildContext is a handle to the location of a widget in the Widget Tree. It is used by methods like Theme.of(context), Navigator.of(context), and MediaQuery.of(context) to look up configuration up the tree.

3. Why do we use setState() in StatefulWidget?

Answer: setState() notifies the Flutter framework that the internal state of the widget has changed. This schedules a call to the build() method, causing the UI to redraw with the new values. Changing a variable without setState() will NOT update the screen.

4. How does State Management work in this AgroManage project?

Answer: We use the ChangeNotifier pattern with AppState. AppState holds lists of products, customers, and invoices. When a method like recordSaleInvoice() is called, it updates the in-memory data and executes notifyListeners(). In main.dart, AnimatedBuilder listens to AppState and automatically rebuilds.

5. What is the difference between Expanded and Flexible in a Row/Column?

Answer: Both are used inside Flex containers (Row/Column). Expanded forces its child to occupy all remaining available space (equivalent to Flexible with fit: FlexFit.tight). Flexible with FlexFit.loose allows the child to only take as much space as it needs up to the maximum available.

6. What is a GlobalKey used for?

Answer: A GlobalKey uniquely identifies a Form widget and gives access to its FormState. It allows calling _formKey.currentState!.validate() to trigger all validator functions across the form, and .save() to save values.

7. Why must TextEditingControllers be disposed?

Answer: Controllers allocate memory and listen to text input streams. If not disposed via controller.dispose() inside the State.dispose() lifecycle method, they cause memory leaks when the screen is popped off the navigation stack.

8. How is the 4-second Splash Screen implemented?

Answer: SplashScreen is a StatefulWidget. Inside initState(), an AnimationController plays an entrance scale/fade animation, and a Timer(const Duration(seconds: 4)) executes a callback onFinish() that flips _showSplash = false in main.dart, seamlessly transitioning to LoginScreen.

9. What is ListView.builder and why is it preferred over a simple ListView?

Answer: ListView.builder creates list items lazily on demand only as they scroll into view on screen. A standard ListView([children]) builds all children at once, which consumes excessive RAM if there are 100+ products or customers.

10. What is pubspec.yaml and what is it used for?

Answer: pubspec.yaml is the project configuration file. It defines the application metadata (name, description, version), Dart SDK constraints, third-party packages (dependencies), development tools (flutter_lints), and local asset paths (images, fonts).

11. How do you pass data from one screen to another screen?

Answer: By passing arguments into the constructor of the destination screen. For example, when opening CustomerDetailScreen(customer: selectedCustomer), the selected customer object is received and stored in final Customer customer.

12. How does the POS screen prevent selling products that are out of stock?

Answer: In pos_screen.dart, before adding an item or confirming an invoice, the code checks if (cartQuantity > product.stock). If true, it displays an error SnackBar ('Insufficient stock for this product') and stops execution.

13. What is an AlertDialog and how is it shown?

Answer: An AlertDialog is a modal popup. It is invoked via showDialog(context: context, builder: (context) => AlertDialog(title: Text(...), content: Text(...), actions: [...])). It is dismissed using Navigator.pop(context).

14. How does Dark Mode work in your application?

Answer: AppTheme defines both lightTheme and darkTheme. AppState contains a ThemeMode property (_themeMode). When the toggle is tapped, it flips between ThemeMode.light and ThemeMode.dark, calls notifyListeners(), and MaterialApp automatically swaps palettes.

15. Why does the examiner prohibit AI tools and require basic code completion?

Answer: To test independent problem-solving skills, understanding of widget structures, and fundamental Dart syntax without relying on automated generation. Only Ctrl + Space basic autocomplete in Android Studio is permitted.